

n8n

Detailed Notes — Workflow Automation Platform

Covers: concepts, architecture, nodes, expressions, deployment, security, use cases, and comparisons with Zapier / Make.

Compiled reference notes — for study and quick lookup.

Contents

1. What is n8n?
2. Core Concepts
3. Architecture & How Execution Works
4. Node Types
5. Triggers
6. Expressions & Data Structure
7. Credentials & Authentication
8. Deployment Options
9. Installation (Docker Quick Start)
10. Key Features
11. Common Use Cases
12. n8n vs Zapier vs Make
13. Pricing Overview
14. Best Practices
15. Limitations / Things to Watch
16. Useful Resources

1. What is n8n?

n8n (pronounced "n-eight-n", short for **nodemation**) is a source-available, extendable workflow automation tool. It lets you connect different apps, APIs, and services together to automate tasks — similar in purpose to Zapier or Make (formerly Integromat), but with a stronger focus on self-hosting, visual + code flexibility, and fair-code licensing.

Workflows are built visually on a canvas by connecting **nodes** (steps) with lines representing the flow of data. Each node performs an action — calling an API, transforming data, sending a message, querying a database, running custom code, etc.

Key facts:

- Built with Node.js/TypeScript; workflows can be exported/imported as JSON.
- Licensed under the **Sustainable Use License** (source-available, "fair-code" — free to use/self-host with some restrictions on reselling/embedding).
- Offers both a visual, low-code builder and the ability to drop into raw JavaScript/Python code when needed.
- Can be self-hosted (Docker, npm, Kubernetes) or used via n8n Cloud (managed SaaS).
- Created in 2019 by Jan Oberhauser; developed by n8n GmbH (Berlin).

2. Core Concepts

Term	Meaning
Workflow	A sequence of connected nodes that defines an automation. Saved as JSON; can be active (running on triggers) or inactive (manual only).
Node	A single building block/step in a workflow — e.g. "Send Email", "HTTP Request", "IF", "Google Sheets". Each node has input/output connections.
Trigger Node	A special node that starts a workflow (e.g. Webhook, Schedule, or an app event like "New row added").
Connection	The line linking one node's output to another node's input, defining execution order and data flow.
Execution	One run of a workflow, triggered manually or automatically. n8n keeps an execution log/history for debugging.
Credential	Securely stored authentication details (API key, OAuth2 token, etc.) that nodes reuse.
Expression	A small piece of code (JavaScript-based, wrapped in {{ }}) used inside node fields to reference or transform data dynamically.
Item	The basic unit of data n8n passes between nodes — roughly one row/object of JSON data. A node usually receives and outputs an array of items.

Sub-workflow	A workflow called from another workflow (via the "Execute Workflow" node), useful for reusable logic.
---------------------	---

3. Architecture & How Execution Works

n8n's core components work together as follows:

- **Editor UI** — the visual, browser-based canvas for building/editing workflows.
- **n8n server (main process)** — hosts the UI, REST API, and (in simple setups) executes workflows.
- **Database** — stores workflows, credentials (encrypted), and execution data. SQLite by default; PostgreSQL/MySQL recommended for production.
- **Trigger listeners** — webhook endpoints, polling schedules, or event listeners that wait to fire a workflow.
- **Worker processes (queue mode)** — in scaled/production deployments, n8n can run in "queue mode" using Redis + Bull/BullMQ, distributing executions across multiple worker instances for horizontal scaling.
- **Webhook process (optional)** — a dedicated process just for receiving incoming webhooks, separated from execution workers for reliability.

Execution flow: a trigger fires → n8n creates an execution → data ("items") flows node by node left to right → each node processes its input items and passes output items to the next node → the execution finishes as Success, Error, or Waiting (e.g. for a human approval step).

4. Node Types

Category	Description / Examples
Trigger nodes	Start workflows: Webhook, Schedule/Cron, Manual Trigger, Email Trigger, App-specific triggers (e.g. "New Slack message").
Regular/Action nodes	Perform an action: send an email, create a record, call an API. Usually app-specific (Slack, Gmail, Notion, Airtable, HubSpot, etc.).
Core/Utility nodes	General-purpose logic: HTTP Request, Set/Edit Fields, IF, Switch, Merge, Split in Batches (Loop), Filter, Code, Function, Wait, No Op.
Transform nodes	Reshape data: Item Lists, Aggregate, Sort, Remove Duplicates, Date & Time.
AI / LangChain nodes	AI Agent, Chat Model, Embeddings, Vector Store, Memory, Tool nodes — for building LLM-powered agents and RAG pipelines directly in workflows.
App integration nodes	Pre-built connectors for 400+ services (Google Workspace, Microsoft 365, Slack, GitHub, Stripe, databases, CRMs, etc.).
Community nodes	Custom nodes built and published by the community, installable into self-hosted instances.

5. Triggers

Every automatic workflow needs exactly one way to start. Common trigger types:

- **Manual Trigger** — run on demand by clicking "Execute Workflow" (good for testing).
- **Webhook** — exposes a unique URL; workflow runs when that URL receives an HTTP request.
- **Schedule / Cron** — runs at fixed intervals (every X minutes, daily at a time, cron expression).
- **App-event triggers** — e.g. "New email in Gmail", "New row in Google Sheets", "New issue in GitHub" — these usually poll the app's API or use the app's own webhooks.
- **Error Trigger** — starts a separate workflow whenever another workflow fails, useful for alerting.
- **Chat Trigger** — starts a workflow from a chat-style interface, commonly used for AI agents.

6. Expressions & Data Structure

n8n passes data between nodes as an array of **items**, where each item is a JSON object typically shaped like:

```
[
  { "json": { "name": "Alice", "email": "alice@example.com" } },
  { "json": { "name": "Bob", "email": "bob@example.com" } }
]
```

Expressions let you reference data dynamically inside any field, using double curly braces:

```
{{ $json.name }}    → current item's "name" field
{{ $node["HTTP Request"].json.body }} → data from a specific earlier node
{{ $now.minus(1, "day") }} → built-in date helper
{{ $json.price * 1.18 }} → inline JS-style calculations
```

For anything more complex, the **Code node** allows full JavaScript (or Python, in newer versions) to loop through and transform items directly.

7. Credentials & Authentication

- Credentials (API keys, OAuth2 tokens, basic auth, etc.) are stored encrypted in the database, separate from workflow JSON, and reused across nodes/workflows.
- n8n supports OAuth1/OAuth2 flows with a built-in redirect handler for apps that require login-based authorization.
- Credentials can be scoped to individual users, shared with team members, or managed centrally in Enterprise/Cloud plans with role-based access control.
- Self-hosted instances require setting an encryption key so stored credentials stay protected at rest.

8. Deployment Options

Option	Notes
n8n Cloud	Fully managed SaaS by n8n GmbH. No infrastructure to maintain; usage-based/tiered pricing; automatic updates.

Self-hosted (Docker)	Most popular self-hosting method. Full control, data stays on your infrastructure, free under the Sustainable Use License for internal use.
Self-hosted (npm)	Install globally via npm and run directly on a server — simplest for quick local testing.
Kubernetes / Helm	For scaled, production-grade deployments using queue mode with multiple workers.
Embedded (via partner license)	n8n can be embedded into another product under a separate commercial/enterprise agreement.

9. Installation (Docker Quick Start)

Minimal self-hosted setup using Docker:

```
docker volume create n8n_data
docker run -it --rm \
  --name n8n -p 5678:5678 \
  -v n8n_data:/home/node/.n8n \
  docker.n8n.io/n8nio/n8n
```

Then open **http://localhost:5678** in a browser to access the editor. For production, pair this with a reverse proxy (HTTPS), PostgreSQL as the database, environment variables for the encryption key, and (optionally) queue mode with Redis for scaling.

10. Key Features

- **Visual workflow builder** with drag-and-drop nodes and live data preview at every step.
- **400+ built-in integrations** plus generic HTTP Request / GraphQL / Webhook nodes to connect to literally any API.
- **Code flexibility** — mix no-code nodes with custom JavaScript/Python via the Code node.
- **Self-hosting & data control** — run entirely on your own infrastructure, useful for privacy/compliance-sensitive workflows.
- **AI-native features** — native LangChain-based nodes for building AI Agents, RAG pipelines, and chatbots.
- **Error handling** — retry logic, error-trigger workflows, and per-node "continue on fail" settings.
- **Version control friendly** — workflows are JSON, so they can be stored/diffed in Git; Enterprise plan adds native Git-based environments.
- **Templates library** — thousands of community-shared, ready-to-use workflow templates.
- **Sub-workflows & reusability** — call one workflow from another to modularize logic.
- **Queue mode & scaling** — horizontally scale execution with worker processes for high-volume automation.

11. Common Use Cases

- **Marketing ops** — sync leads between forms, CRM, and email tools; automate drip campaigns.

- **Sales** — enrich new leads with third-party data, auto-create CRM records, notify reps on Slack.
- **DevOps** — CI/CD notifications, auto-deploy triggers, monitoring alerts routed to Slack/Teams/PagerDuty.
- **Data sync / ETL** — move and transform data between databases, spreadsheets, and SaaS apps on a schedule.
- **Customer support** — auto-tag and route tickets, generate AI-drafted responses, escalate based on rules.
- **Internal tooling** — build lightweight internal apps/back-office automations without a full dev cycle.
- **AI agents** — orchestrate LLM calls, tools, memory, and vector search into multi-step autonomous agents.

12. n8n vs Zapier vs Make

Aspect	n8n	Zapier	Make (Integromat)
Hosting	Self-host (free) or Cloud	Cloud only	Cloud only
Pricing model	Flat/execution-based; free self-hosted	Per task, gets costly at scale	Per operation
Custom code	Full JS/Python via Code node	Limited (Code step, paid tier)	Limited (custom functions)
Complex logic (loops, branching)	Strong, visual + code	Basic, more rigid	Strong, visual
Integrations count	400+ built-in + generic HTTP	6000+ (largest catalog)	1000+
Best for	Technical teams, self-hosting, complex workflows	Simple, quick setups	Multiple complex automations, mid-technical users

13. Pricing Overview

Pricing details change over time, so always check n8n's official pricing page for current numbers. As a general structure:

- **Self-hosted (Community)** — free to use under the Sustainable Use License; you cover your own infrastructure.
- **Cloud plans** — tiered subscription plans (e.g. Starter/Pro) based on number of active workflows and workflow executions, billed monthly or annually.
- **Enterprise** — custom pricing; adds SSO, RBAC, environments/Git sync, dedicated support, and self-hosting with enterprise features.

14. Best Practices

- Name nodes descriptively (avoid leaving default names like "HTTP Request1") for easier debugging.
- Use the **Set/Edit Fields** node to normalize data shape early in a workflow.
- Break large workflows into **sub-workflows** for reusability and readability.
- Add an **Error Trigger workflow** to catch failures and alert the team (Slack/email).
- Use **pinned data** during development to avoid re-triggering real API calls while testing.
- Store secrets only in **Credentials**, never hard-coded in node parameters.

- For production, use PostgreSQL instead of SQLite, and enable queue mode if execution volume is high.
- Version-control workflow JSON exports (or use Enterprise Git environments) for change tracking.

15. Limitations / Things to Watch

- Self-hosting requires ops know-how (Docker, database, backups, updates, security patching).
- The Sustainable Use License restricts reselling n8n itself or offering it as a hosted service to third parties.
- Some app integrations lag behind newer API features compared to larger platforms like Zapier.
- Steeper learning curve than pure no-code tools if you need to use expressions or the Code node.
- Very large/high-volume workflows need careful architecture (queue mode, batching) to avoid performance issues.

16. Useful Resources

- Official docs: docs.n8n.io
- Workflow template library: n8n.io/workflows
- Community forum: community.n8n.io
- GitHub repository: github.com/n8n-io/n8n

End of notes.